

Лабораторная 8. Хеш-таблица с отдельными цепочками

Зачем нужна хеш-таблица

Ассоциативная структура связывает ключ со значением: например, `user_0015` с числом посещений. Доступ выполняется по ключу, а не по порядковому индексу. В данной работе поддерживаются строковые ключи и произвольные значения. Нужно реализовать `put`, `get`, `delete`, размер, рост таблицы и исследовать коллизии.[1]

Внутри находится массив бакетов. Бакет — контейнер для записей, которым достался одинаковый индекс. Функция `hash(key)` получает целое число, а остаток от деления на число бакетов выбирает индекс. В каждой цепочке хранятся пары `[key,value]`. Готовый `dict` внутри `HashTable` не используется; списки служат контейнерами отдельных цепочек.

Коллизия — нормальная ситуация

Разные ключи могут попасть в один бакет. Это не означает, что они равны: совпал только индекс. Поэтому `get` просматривает цепочку и сравнивает исходные ключи. При `put` сначала ищется существующий ключ; его значение заменяется без увеличения размера. Если ключа нет, добавляется новая пара.

Например, при четырёх бакетах ключи с хешами 5 и 9 оба попадут в бакет 1. Там одновременно могут храниться две записи. Удаление одной не должно повреждать вторую. В этой реализации `get` и `delete` для отсутствующего ключа выбрасывают `KeyError`, как доступ и удаление через `[]` у `dict`. Значение `None` разрешено и отличается от отсутствующего ключа.

Полиномиальная хеш-функция

Начинаем с $h=0$ и для каждого символа выполняем $h=(131 \cdot h + \text{ord}(\text{symbol})) \bmod 1\,000\,000\,007$. Важны и символы, и порядок их расположения. Например, у `ab` и `ba` разные веса первого и второго символов; простая сумма кодов этого не различает.

Хеш должен быть детерминированным для неизменного ключа, учитывать его содержимое, вычисляться достаточно быстро и распределять используемые ключи по бакетам без сильной концентрации. Это не криптографическая функция: она не предназначена для паролей или защиты от специально подобранных атак. Для учебного опыта требуется распределение обычных строковых ключей.[2]

Плохая функция в опыте возвращает код первого символа. Все ключи `user_XXXX` начинаются с `u`, поэтому вся таблица оказывается одной длинной цепочкой. Увеличение числа бакетов не исправит такой дефект: хеши по-прежнему равны.

Коэффициент загрузки и rehash

Коэффициент $\alpha = n/V$, где n — число уникальных записей, V — число бакетов. При достаточно равномерном распределении это средняя длина цепочки. Если $n=600$, $V=1000$, то $\alpha=0,6$; не все цепочки имеют длину 0 или 1, но среднее по всем бакетам равно 0,6.

В обычном режиме после добавления нового ключа проверяется α . Если $\alpha > 0,75$, число бакетов удваивается и все записи распределяются заново. Индекс зависит от V , поэтому

просто увеличить массив пустыми бакетами недостаточно: старые записи могли бы оказаться в неправильных местах.

При $V=8$ шесть ключей дают $\alpha=0,75$ и роста ещё нет. Седьмой ключ даёт $0,875$; после расширения $V=16$ и $\alpha=7/16$. Замена значения существующего ключа не увеличивает n и не вызывает рост. Удаление не уменьшает число бакетов: уменьшение ёмкости не требовалось условием и здесь не реализовано.

Сколько стоят операции

При равномерном распределении поиск по цепочке занимает ожидаемое $O(1+\alpha)$. Если ключи имеют ограниченную длину и α поддерживается ограниченным, `get/delete` ожидаемо $O(1)$, а `put` с удвоением — ожидаемо амортизированно $O(1)$. Один `put`, запустивший `rehash`, стоит $O(n)$ по числу записей.

Для строк длины L нужно учитывать вычисление хеша $O(L)$, а сравнение двух строк в худшем случае тоже $O(L)$. Более точная верхняя оценка поиска по цепочке — $O(L(1+\alpha))$ в средней модели и $O(nL)$ при полном столкновении. Обычная запись $O(1)$ подразумевает ограниченную стоимость ключа, а не бесплатное чтение сколь угодно длинной строки.

В худшем случае все n ключей находятся в одной цепочке: `get`, `delete` и `put` без расширения требуют $O(n)$ проверок. `Rehash` при удвоении выполняется не на каждом добавлении. Сумма объёмов перераспределений растёт геометрически и даёт $O(N)$ для N вставок при ограниченной длине ключей, поэтому стоимость роста амортизируется.

Удаление из цепочки Python-списка сдвигает её хвост. Это тоже $O(\text{длины цепочки})$, поэтому не меняет порядок оценок для отдельного цепочечного хеширования. Настоящий связный список внутри бакета имел бы другие постоянные затраты.

Цепочки и открытая адресация

При цепочках конфликтующие ключи хранятся в отдельных контейнерах. α может превышать 1, корректность от этого не теряется. Удаление не требует специальных «надгробий», но есть дополнительные контейнеры и косвенные обращения к памяти.

При открытой адресации записи находятся в самом массиве; при занятой позиции ищется другая по последовательности проб. Нужны свободные ячейки, поэтому α должен оставаться меньше 1. Доступ часто лучше использует кэш, однако удаление и высокое заполнение требуют аккуратной обработки. Встроенный `dict` использован как практический эталон поведения и времени, а не как реализация отдельных цепочек.

Как устроен эксперимент

```
python -m pytest labs/lab08 -q
python -m labs.lab08.experiment --variant 15
```

Ключи берутся из официального генератора варианта 15. Для гистограмм используются уникальные строки на одинаковом числе бакетов и сравниваются полиномиальная и плохая функции. Высота столбца — число бакетов данной длины, а не число ключей. Нулевая длина важна: она показывает пустые бакеты.

Для зависимости времени от α автоматический рост временно отключается специальным параметром `auto_resize=False`. Иначе нельзя было бы независимо исследовать $\alpha > 0,75$: таблица немедленно расширилась бы. В обычном интерфейсе по умолчанию `auto_resize=True`, и порог соблюдается.

Измеряются успешное чтение, неуспешное чтение, обновление значения и вставка нового ключа. У нового ключа начальная и конечная α различаются, потому что один замер включает 64 вставки; обе величины записаны в CSV. Для dict α означает размер той же нагрузки относительно 1024, а не внутренний коэффициент загрузки dict — его реальная ёмкость здесь не контролируется.

Таблица для каждого повтора создаётся до таймера. Поэтому в опыте фиксированной α не измеряются стоимость первоначального построения и rehash. Их корректность и порог роста проверяются отдельными тестами. Неуспешное чтение включает создание и перехват KeyError у обоих интерфейсов.

Проверка инвариантов

Каждый ключ должен находиться в бакете $\text{hash}(\text{key})\%B$; один ключ встречается ровно один раз; сумма длин цепочек равна size. При включённом росте после завершения put должно быть $\alpha \leq 0,75$. Эти свойства проверяет validate.

Три серии по 4000 случайных операций сравниваются с dict: с хорошей функцией, плохой функцией и хешем, всегда возвращающим 0. Последний сценарий принудительно сталкивает все ключи и проверяет, что коллизии влияют только на скорость. Отдельно проверяются пустая строка, None как значение, замена существующей записи, точный порог 0,75 и многократное расширение.

Вопросы на защите и ответы

- 1. Какой должна быть хорошая хеш-функция?** Одинаковый ключ должен давать одинаковый хеш, вычисление должно быть недорогим, а ключи реального набора — распределяться достаточно равномерно. Первый символ не годится для user_XXXX: он одинаков у всех.
- 2. Как влияет α и зачем порог 0,75?** Более высокая α обычно означает более длинные цепочки. Порог ограничивает среднюю длину в обычной работе. Для цепочек 0,75 — выбранная политика производительности, не граница корректности.
- 3. Чем цепочки отличаются от открытой адресации?** Цепочки держат несколько ключей в отдельном контейнере бакета; открытая адресация ищет другую позицию того же массива. У них разные расходы памяти, локальность и правила удаления.
- 4. Какова средняя и худшая сложность?** При ограниченной длине ключей ожидаемо $O(1+\alpha)$, при ограниченной α — $O(1)$; худший случай $O(n)$. Для вставки нужно отдельно указать амортизацию rehash и стоимость единственного расширения $O(n)$.
- 5. Почему rehash переносит все ключи?** Изменился делитель B в вычислении индекса. Без перераспределения поиск начнёт смотреть в новый бакет, а запись останется в старом.
- 6. Почему обновление не увеличивает size?** Число уникальных ключей не изменилось. Таблица связывает один ключ с одним текущим значением.
- 7. Можно ли отличить None от отсутствия?** Да: существующая запись возвращает None, отсутствующая вызывает KeyError. Если бы get всегда возвращал None при отсутствии, понадобился бы отдельный признак наличия.
- 8. Почему dict значительно быстрее собственной таблицы?** Его операции реализованы на низком уровне и оптимизированы. Эксперимент показывает конкретные затраты, а не доказательство иной асимптотики.

Источники

[1] МТУСИ. КИМ-08, 2026.

[2] Robert Sedgewick, Kevin Wayne. Hash Tables, материалы авторов Algorithms, 4th edition.

Отчёт по ЛР 8. Хеш-таблица

Цель и выполненная работа

Реализованы алгоритмы и проверки, описанные в конспекте. Эксперимент выполнен на данных варианта 15, seed 45. В таблицах приведены фактические результаты запуска; время указано в секундах, если не оговорено иное.

Условия и методика

CPU: 12th Gen Intel(R) Core(TM) i7-12650H. ОС: Windows-11-10.0.26200-SP0. Python 3.12.12, NumPy 2.5.3, Matplotlib 3.11.1. Дата и время UTC: 2026-09-10T18:30:55.434837+00:00.

Один прогрев и пять повторов на точку, итог — медиана `time.perf_counter`. Ввод, генерация и подготовка свежего состояния исключены из таймера. Фоновые процессы и питание не контролировались; задержки рабочего компьютера могут влиять на отдельные точки. Контрольные суммы данных находятся в `environment.json`.

Результаты и их объяснение

Хеш	Длина цепочки	Бакетов	α
polynomial	0	11085	0.610352
polynomial	1	1972	0.610352
polynomial	2	2124	0.610352
polynomial	3	1032	0.610352
polynomial	4	171	0.610352
first_character	0	16383	0.610352
first_character	10000	1	0.610352

При 10000 ключах и 16384 бакетах $\alpha \approx 0,61035$. У полиномиальной функции максимальная цепочка имеет длину 4; функция первого символа помещает все 10000 ключей в один бакет. Распределение полиномиальной функции не идеально равномерно, но концентрация на этом наборе намного меньше.

Реализация	Операция	α до	α после	с/операцию
polynomial	get_hit	4	4	1.47031e-06
polynomial	get_miss	4	4	1.84687e-06
polynomial	put_update	4	4	1.3125e-06
polynomial	put_new	4	4.0625	1.75312e-06
first_character	get_hit	4	4	7.10688e-05
first_character	get_miss	4	4	0.000147192
first_character	put_update	4	4	6.71984e-05
first_character	put_new	4	4.0625	0.000134742
dict	get_hit	4	4	1.76562e-07

Реализация	Операция	α до	α после	с/операцию
dict	get_miss	4	4	3.5625e-07
dict	put_update	4	4	1.98438e-07
dict	put_new	4	4.0625	1.90625e-07

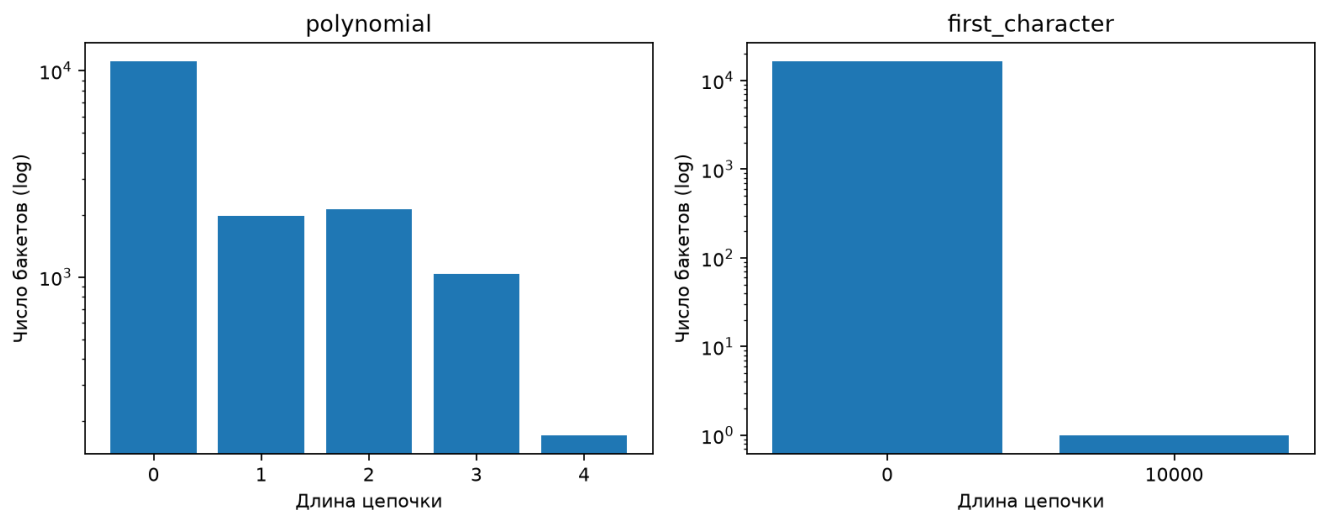
В контролируемом опыте авторасширение отключено, чтобы независимо менять α . Обычный режим таблицы по умолчанию расширяется после $\alpha > 0,75$, это проверено тестами. При put_new коэффициент меняется в течение 64 вставок; начальное и конечное значения сохранены. Для dict по горизонтали указана внешняя нагрузка $n/1024$, а не его внутренняя α . Стоимость rehash не входит в эти фиксированные по ёмкости замеры.

Проверка и вывод о применимости

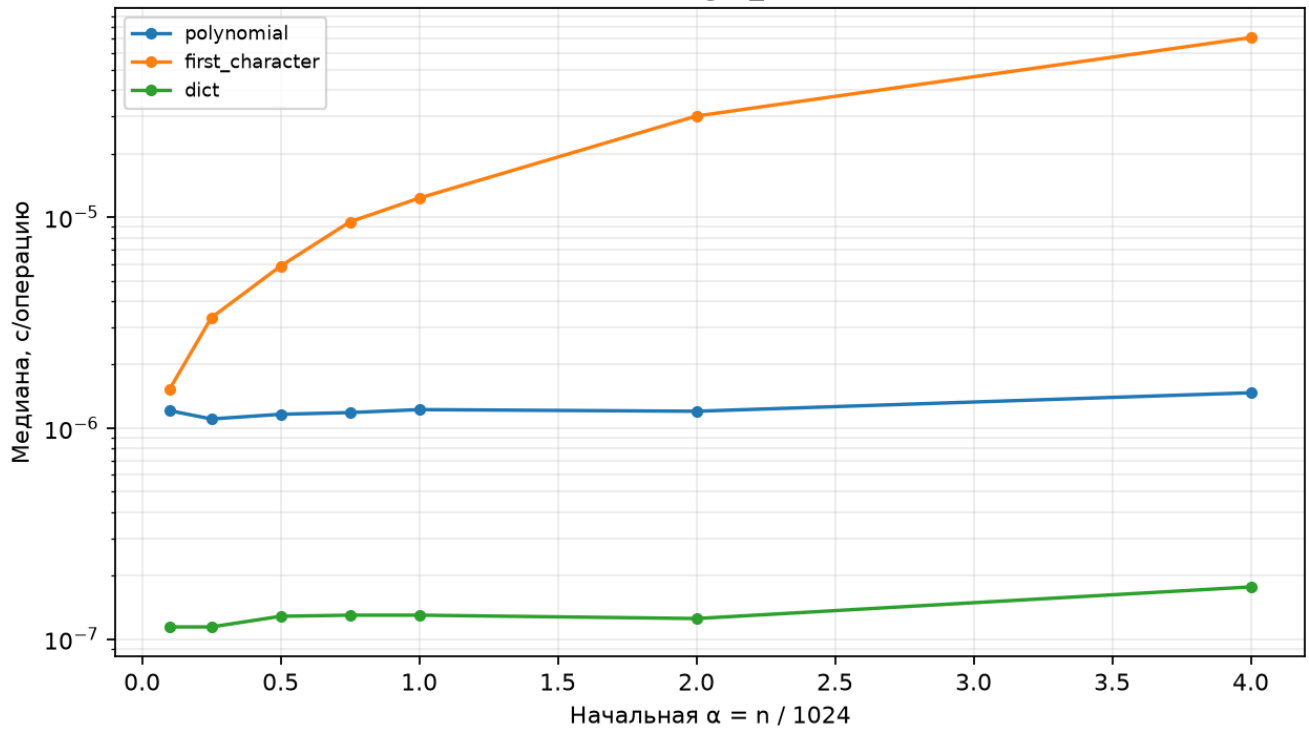
Проверки находятся в test_lab.py. Запуск: `python -m pytest labs/lab08 -q`. Применены граничные случаи, инварианты и независимые эталоны; состав тестов подробно разобран в конспекте. Проверки всего комплекта также сохранены в verification.txt в корне репозитория.

Вывод: принять для учебного использования в рамках явно описанных контрактов. Измерения подтверждают основные тенденции роста; ограничения реализаций и сравнения указаны выше и в конспекте. Автоматическая проверка не оценивает готовность к устной защите.

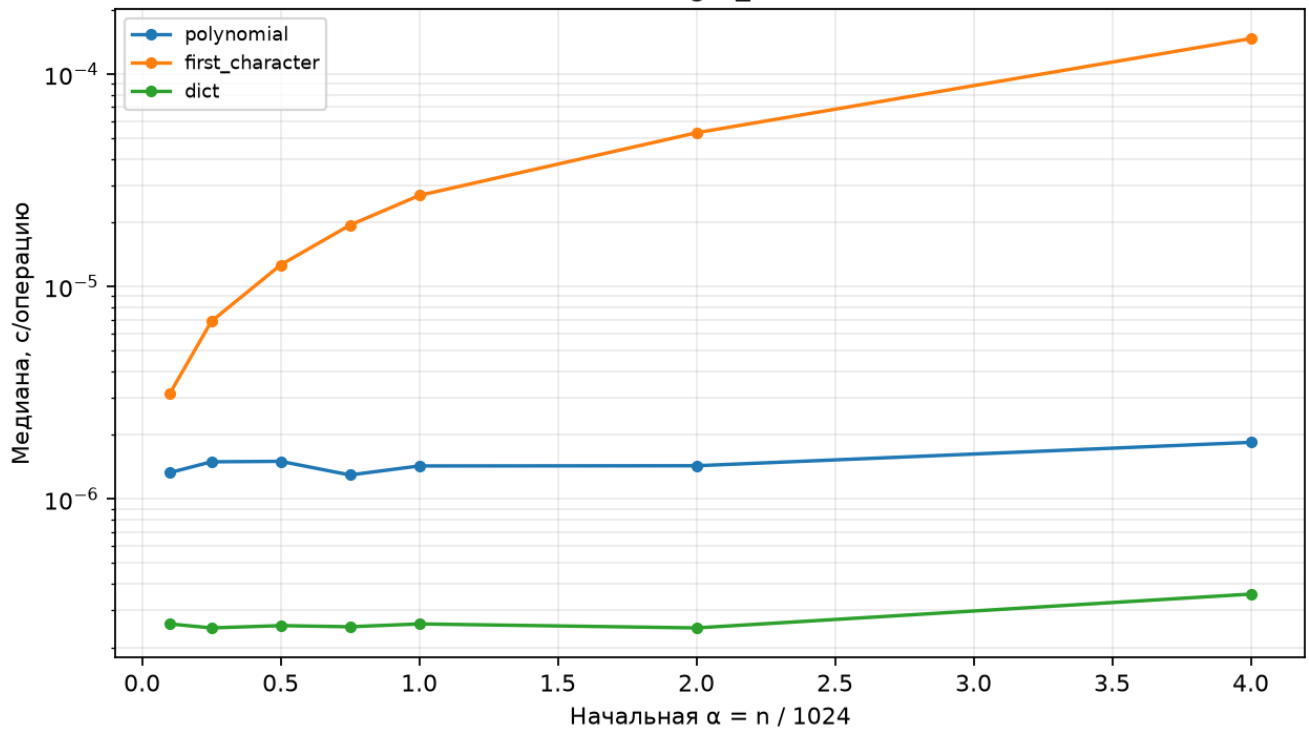
Графики

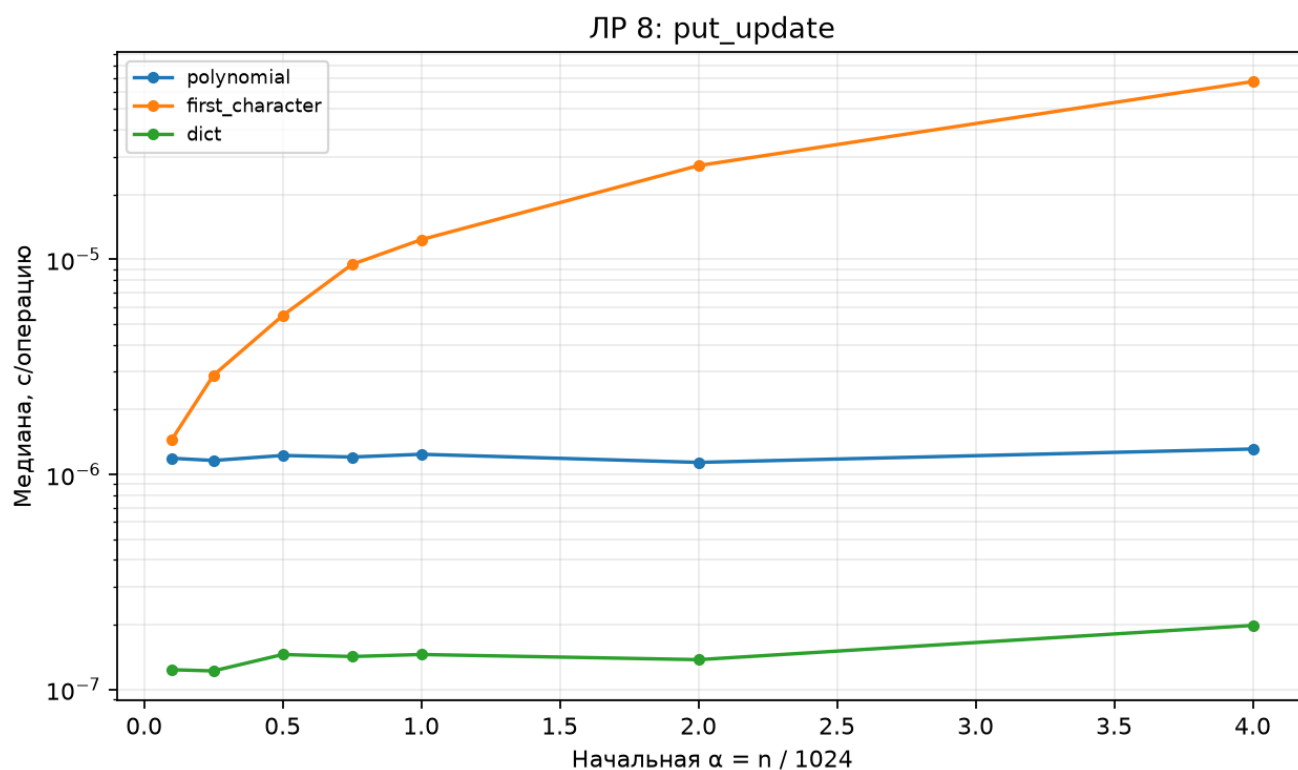
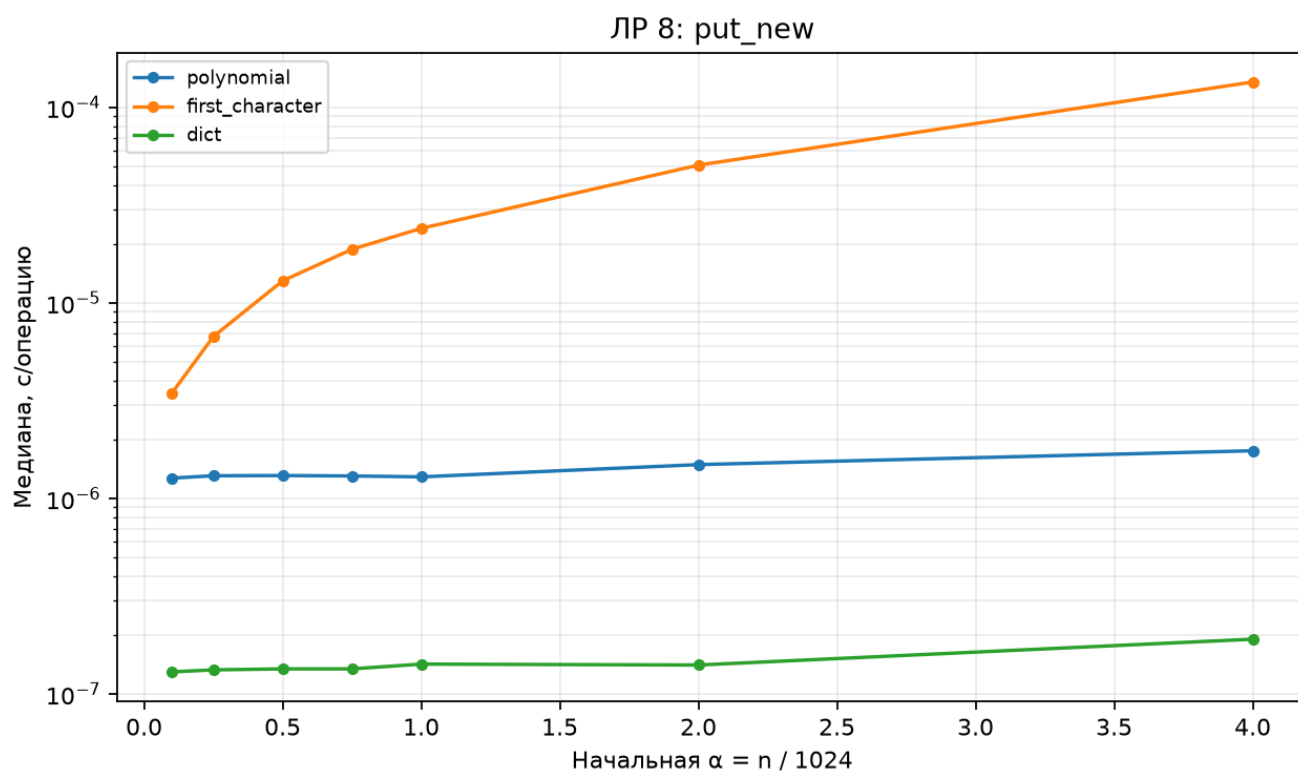


ЛП 8: get_hit



ЛП 8: get_miss





Исходные данные отчёта

CSV содержит медиану и все пять длительностей trial_1...trial_5. Вспомогательные таблицы без времени содержат точные счётчики или свойства структуры.

- chains.csv
- operations.csv

- Паспорт окружения и контрольные суммы